HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

Design and Implementation of a Deep Learning-Based Optical Character Recognition System for Vietnamese Document Processing

TRỊNH HUYNH SƠN
son.th225526@sis.hust.edu.vn

Program: Data Science & AI

Supervisor: Associate Professor Nguyễn Thị Kim Anh

Department: Computer Science

School: School of Information and Communications Technology

HANOI, 06/2026

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

Thesis title

Trịnh Huynh Sơn
son.th225526@sis.hust.edu.vn

Program: Data Science AI

Supervisor: Associate Professor Nguyễn Thị Kim Anh _____

Signature

Department: Computer Science

School: School of Information and Communications Technology

HANOI, 06/2026

ACKNOWLEDGMENTS

First and foremost, I would like to express my deepest gratitude to my supervisor, Associate Professor Nguyễn Thị Kim Anh, and the faculty at Hanoi University of Science and Technology. Her invaluable guidance, rigorous feedback, and continuous encouragement were instrumental in shaping the research methodology and technical approach of this thesis.

I am profoundly thankful to my colleagues and friends at FSI. Their insightful suggestions and generous support in providing the necessary testing infrastructure were crucial for evaluating the pipeline under real-world conditions.

Furthermore, I extend my heartfelt thanks to my family for their unwavering emotional support during the most challenging debugging phases. Finally, I want to acknowledge my own perseverance and dedication, which drove me to successfully complete this system.

Trịnh Huynh Sơn

ABSTRACT

In the era of digital transformation, converting physical documents into structured digital data has become an essential task for optimizing business processes. Although traditional Optical Character Recognition (OCR) systems have been widely adopted, they primarily focus on extracting plain text and often exhibit significant limitations when processing documents with complex layouts, such as multi-column formats and tables. These limitations frequently disrupt the natural reading order and result in the loss of semantic information. Furthermore, the complexity of the Vietnamese writing system, particularly its extensive use of diacritical marks, poses additional challenges that reduce the accuracy of conventional OCR models.

To address these challenges, this project adopts an integrated approach that combines advanced Deep Learning models within an Event-Driven Architecture (EDA). This approach enables the system to understand the overall visual structure of a document before performing detailed text recognition, while the distributed architecture ensures scalability, reliability, and efficient processing of large data streams.

The project successfully designs and implements a fully automated Document AI pipeline. The proposed solution integrates a YOLO-based layout analysis model for semantic region detection, the DBNet model from PaddleOCR for text line localization, and the fine-tuned VGG-Transformer architecture of VietOCR for highly accurate Vietnamese text recognition. All recognition services are containerized using Docker and orchestrated asynchronously through Apache NiFi and Apache Kafka. In addition, a heuristic geometric merging algorithm is employed to restore the original reading order of the document.

The primary contribution of this project is the realization of a practical end-to-end MLOps system capable of real-world deployment. The resulting solution can independently extract tabular data, effectively handle image noise, and convert complete document contents into well-structured JSON and Markdown outputs. This capability directly addresses the growing demand for advanced document digitization in modern organizations.

Student

(Signature and full name)

Trịnh Huynh Sơn

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION 1

1.1 Motivation 1

1.2 Objectives and scope of the graduation thesis 2

1.3 Tentative solution 3

1.4 Thesis organization 4

CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS 6

2.1 Status survey 6

2.2 Functional Overview 7

2.2.1 General system context 7

2.2.2 Detailed functional decomposition 8

2.2.3 Pipeline workflow 8

2.3 Functional description 9

2.3.1 Image Preprocessing 9

2.3.2 Layout Analysis 9

2.3.3 Text Cropping 9

2.3.4 Vietnamese Character Recognition 10

2.3.5 Data Reconstruction 10

2.4 Non-functional requirements 10

CHAPTER 3. THEORETICAL BACKGROUND AND TECHNOLOGIES 11

3.1 Convolutional Neural Networks (CNN) 11

3.2 Document Layout Analysis Model 11

3.3 Text Detection Model 12

3.4 Vietnamese Character Recognition Model 13

3.5 Event-Driven Architecture and Message Broker 14

3.6 Object Storage and Virtualization Technology 15

CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION..... 16

4.1 Architecture design 16

4.1.1 Software architecture selection 16

4.1.2 Overall design..... 16

4.1.3 Detailed package design 18

4.2 Detailed design..... 19

4.2.1 Interface and API design..... 19

4.2.2 Layer design 19

4.2.3 Database and Message Schema design 20

4.3 Application Building..... 20

4.3.1 Libraries and Tools..... 20

4.3.2 Achievement..... 21

4.3.3 Illustration of main functions 21

4.4 Model Training and Evaluation 22

4.4.1 Experimental setup and dataset preparation 22

4.4.2 Empirical results and model comparison 23

4.5 Testing..... 23

4.6 Deployment 24

CHAPTER 5. SOLUTION AND CONTRIBUTION 25

5.1 Dataset optimization and vocabulary space expansion..... 25

5.2 Proposed specific Data Augmentation process..... 26

5.3 Hard Negative Mining optimization..... 26

5.4 Pipeline throughput and system stability evaluation 27

5.5 Error Analysis and proposed improvements..... 27

CHAPTER 6. CONCLUSION AND FUTURE WORK 29

6.1 Conclusion..... 29

6.2 Future work..... 30

REFERENCE 33

LIST OF FIGURES

Figure 1.1 Traditional Manual Process 2
Figure 1.2 Propose Document AI pipeline 4

Figure 2.1 General system context diagram showing interactions with external systems 8
Figure 2.2 Event-driven data pipeline workflow across microservices . 9

Figure 3.1 The single-stage object detection architecture of YOLO . . . 12
Figure 3.2 The architecture of DBNet with the Differentiable Binarization module 13
Figure 3.3 The Publish-Subscribe messaging model of Apache Kafka . 14

Figure 4.1 Overall package dependency diagram of the Document AI system 17
Figure 4.2 Detailed design of the AI Workers package 18
Figure 4.3 Sequence Diagram of the OCR Pipeline execution flow . . . 20
Figure 4.4 Structure of the output JSON file with coordinates and classification labels 22

LIST OF TABLES

Table 2.1 Comparison of current OCR solutions against business constraints 7

Table 4.1 List of utilized libraries and tools 21

Table 4.2 Empirical comparison of recognition models on the validation dataset 23

Table 5.1 Top 10 most frequently confused character pairs in the model 27

LIST OF ABBREVIATIONS

Abbreviation	Full Expression
API	Application Programming Interface
CER	Character Error Rate
CNN	Convolutional Neural Network
DMS	Document Management System
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
KIE	Key Information Extraction
LLM	Large Language Model
MLOps	Machine Learning Operations
OCR	Optical Character Recognition
OOM	Out Of Memory
TSR	Table Structure Recognition
WER	Word Error Rate
YOLO	You Only Look Once

CHAPTER 1. INTRODUCTION

Chapter 1 serves as an introduction, presenting the background and rationale for this graduation thesis. This chapter details the research motivation, elucidates the limitations of existing solutions, and subsequently defines the objectives and scope of the study. Following this, the chapter outlines the technological orientation and the technical solutions selected to address the problem. Finally, the overall layout of the entire thesis report is summarized to help readers navigate the flow of information in the subsequent chapters.

1.1 Motivation

In the context of robust digital transformation, converting data from physical documents such as printed materials, contracts, invoices, and administrative texts into structured digital formats is an urgent requirement. The traditional manual data entry process reveals many inherent weaknesses, such as being time-consuming, incurring high operational personnel costs, and carrying the risk of errors during operation. To automate this process, Optical Character Recognition (OCR) systems were developed and have become an indispensable component in enterprise information systems.

However, the practical problem of document digitization does not stop at mere text extraction. Administrative and corporate documents often possess complex presentation structures, including multiple data columns, tables, illustrative images, signatures, and legal seals. The application of traditional OCR tools often disrupts the natural reading flow, destroys the spatial structure of the page, and mixes regular text with content located within tables. Consequently, the output information loses its original semantic meaning, hindering the automation of subsequent analytical and business processing stages.

Furthermore, the Vietnamese language, with its diverse diacritic system and distinct typographical structure, poses a significant challenge for accurate recognition, especially in documents with low print quality, blurred scans, or noise. The core issue currently is building a comprehensive digitization system capable of multi-dimensional visual analysis to comprehend document layouts, combined with high-accuracy natural language recognition capabilities. Once this issue is resolved, organizations can automatically extract and integrate data directly into Enterprise Resource Planning (ERP) or Document Management Systems (DMS), thereby optimizing workflows, reducing costs, and expanding applicability across various sectors such as public administration, finance, legal, and national data archiving.

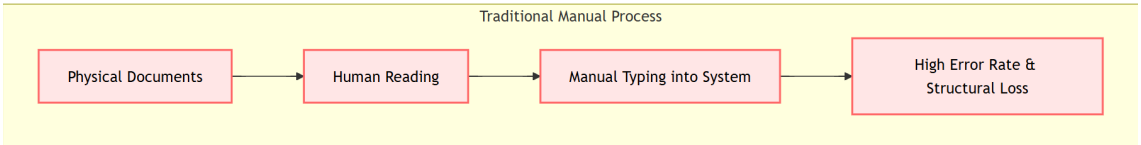


Figure 1.1: Traditional Manual Process

1.2 Objectives and scope of the graduation thesis

Currently, the document digitization problem is typically addressed through two main groups of solutions in the market. The first group utilizes traditional open-source libraries (such as Tesseract). Although easy to deploy locally, these tools often overlook spatial structure analysis, resulting in output data that comprises only raw text blocks, lacks cohesion, and fails to handle multi-column formats or complex tables. The second group employs commercial cloud-based recognition services from major technology corporations. These solutions offer high accuracy and robust layout recognition capabilities but come with substantial operational costs, require continuous internet connectivity, and pose potential security risks to an organization's internal information. Moreover, some commercial services are not fully optimized for the specific formatting characteristics of specialized administrative documents in Vietnam.

Based on an analysis of the trade-offs between security, cost, and the ability to preserve document structure, this thesis aims to solve the problem of constructing a Document AI Pipeline tailored for the Vietnamese language, capable of operating independently on internal server infrastructure. The project will overcome the structural loss shortcomings of current open-source systems by modularizing the processing pipeline into specialized subsystems.

On this basis, the developed software aims to provide core functionalities including: receiving image documents in PDF or scanned formats; automatically analyzing and bounding layout components (paragraphs, headers, tables, lists); and precisely localizing and extracting Vietnamese text. Notably, the system features a function to isolate non-linguistic components (such as tables and images) for storage as static assets, preventing interference with the text flow. Finally, the software reconstructs the data according to the natural reading order and exports it into a standard JSON format and Markdown text. The scope of the thesis focuses on designing the software architecture, integrating, optimizing training datasets, and deploying existing deep learning models into a practical system, rather than researching entirely new neural network architectures.

1.3 Tentative solution

To achieve the established objectives, the solution orientation of the thesis is to integrate advanced Deep Learning models in Computer Vision and Natural Language Processing (NLP) upon an Event-Driven Architecture software platform.

Specifically, the solution divides the processing pipeline into four sequential stages. The layout analysis stage utilizes the YOLO object detection model to recognize page structure. The text detection stage applies PaddleOCR's DBNet network to extract text lines. The character recognition stage employs VietOCR's VGG-Transformer architecture, specifically fine-tuned on a synthetic Vietnamese text dataset. The communication between these models is coordinated entirely asynchronously via Apache NiFi and the Apache Kafka messaging system, with MinIO serving as the intermediate object storage. The final data is consolidated using a local geometric algorithm (Heuristic Merge). The entire architecture is containerized using Docker technology to ensure consistency across deployment environments.

The primary contribution of this thesis is the successful realization of an applied software system adhering to MLOps standards, comprehensively solving the problem from input to output (End-to-End). The achieved result is a stable internal document digitization platform capable of independently separating tables, robustly handling image noise, attaining high performance in Vietnamese text recognition, and exporting data into a strictly structured JSON format without relying on external API services.

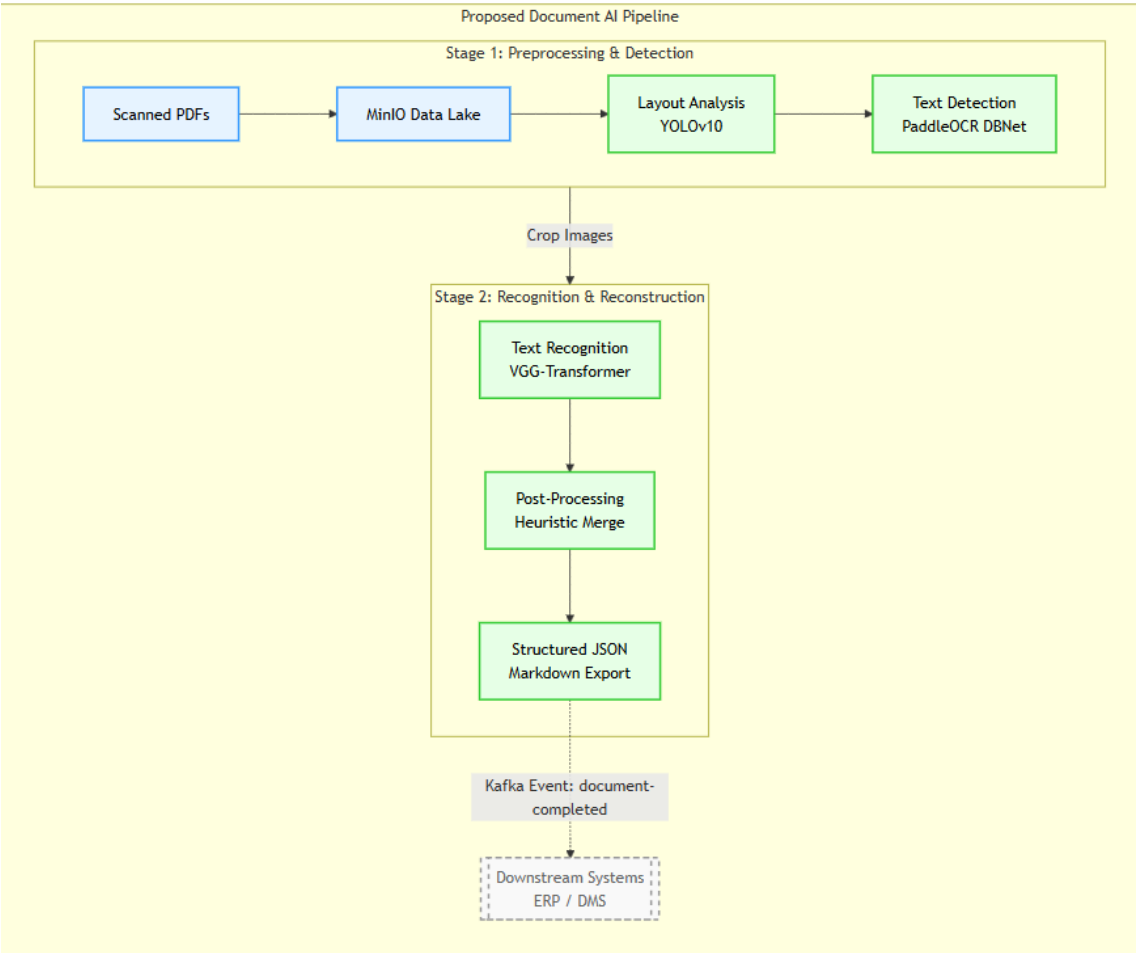


Figure 1.2: Propose Document AI pipeline

1.4 Thesis organization

The remainder of this graduation thesis report is organized as follows.

Chapter 2 presents a detailed survey of the current business state and an analysis of system requirements. In this chapter, the functional and non-functional requirements of the software are clearly defined. Based on these requirements, a comprehensive digitization business process is proposed, serving as a solid foundation for designing the software architecture in subsequent phases.

In Chapter 3, I introduce the theoretical background and core technologies applied to build the system. The content covers the mathematical foundations and architectures of deep neural networks such as YOLO, DBNet, and the hybrid VGG-Seq2Seq network. Additionally, concepts regarding event-driven software architecture, message broker systems, and container virtualization technology are analyzed in detail.

Chapter 4 describes the entire process of designing, implementing, and evaluating the system in practice. In-depth technical issues regarding data orchestra-

tion pipeline configuration, computational resource limitation techniques for performance optimization, dataset preparation, and recognition model fine-tuning are clearly presented. The text reconstruction post-processing algorithm and the output JSON data structure are also explained, accompanied by a performance evaluation through empirical results.

Chapter 5 discusses the overall solution and the core contributions of the thesis. The chapter focuses on evaluating the maturity of the end-to-end MLOps architecture, analyzing the superiority of its approach to handling complex document structures compared to legacy methods, and clarifying the practical applicability of the system in an enterprise production environment.

Chapter 6 summarizes all the results achieved during the research and execution of the thesis. Based on these results, the chapter also points out the remaining limitations of the current system and proposes directions for future software development and improvement.

Chapter 1 has established the foundation for the entire thesis by clarifying the urgency of the structured document digitization process in current organizations. By analyzing the limitations of traditional OCR solutions, the thesis has identified the objective of designing a system for automatic layout recognition and accurate Vietnamese language extraction. The solution, combining deep learning models and an event-driven architecture, has been outlined in overview. Based on this orientation, the subsequent Chapter 2 will delve into surveying the current state and analyzing specific requirements in detail to construct the business workflow for the system.

CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS

Chapter 1 established the necessity of developing a document digitization system capable of preserving spatial structures and accurately recognizing the Vietnamese language. Building upon those guidelines, Chapter 2 focuses on a detailed survey of current existing recognition tools, thereby analyzing and pinpointing the limitations that need to be addressed. Based on the survey results, this chapter defines the functional and non-functional requirements of the system, constructs an overall context model, presents functional decompositions, and models the end-to-end digitization business process. The specifications outlined in this chapter serve as the architectural design foundation for the entire software system in subsequent stages.

2.1 Status survey

The current state of Optical Character Recognition (OCR) technology is evaluated across two core solution groups: open-source tools and commercial cloud-based recognition services (Cloud APIs).

Regarding the open-source group, Tesseract OCR and the vanilla configuration of PaddleOCR are the most widely used libraries. The primary advantage of this group is that they are entirely free and support on-premise deployment, ensuring absolute data security. However, when dealing with complex administrative documents that feature multi-column layouts or tables, these tools typically perform text scanning in a linear stream. This approach breaks down tabular structures, merges content across different columns, and fails to isolate graphical entities (seals, figures), resulting in a complete loss of the original semantic meaning in the output data. Furthermore, the Vietnamese vocabulary of these libraries often lacks sufficient coverage of specialized characters in documents.

As for the cloud services group (such as Google Cloud Vision, AWS Textract, or APIs provided by FPT.AI and Viettel), these systems deliver exceptionally high recognition accuracy and relatively robust layout extraction capabilities. Nevertheless, their primary drawback is the operational cost, which scales proportionally with the volume of processed document pages. Additionally, the mandatory uploading of internal documents and non-disclosure contracts to third-party servers violates the stringent data security standards enforced by many enterprises and state organizations.

Based on the evaluation of the pros and cons of these two solution groups, this

thesis proposes the construction of a self-hosted Document AI Pipeline. The software to be developed must satisfy four critical features: (i) the capability to operate completely offline on internal server infrastructure; (ii) automatic document layout analysis to detect and isolate tables and graphics into independent static assets; (iii) the provision of a specialized processing pipeline for Vietnamese text recognition; and (iv) data reconstruction and export into structured JSON formats containing spatial coordinates alongside standardized Markdown text.

Criteria	**Open-source (e.g., Tesseract)**	**Cloud APIs (e.g., AWS Textract)**	**Proposed Document AI Pipeline**
Deployment Model	On-premise	Cloud-based	**On-premise (Dockerized)**
Operational Cost	Free	High (Pay-per-page)	**Free (Self-hosted)**
Data Privacy	Absolute	High risk	**Absolute**
Layout Preservation	Poor (Linear scan)	Excellent	**Excellent (YOLO Object Detection)**
Vietnamese Accuracy	Average	High	**High (VGG-Transformer Fine-tuned)**

Table 2.1: Comparison of current OCR solutions against business constraints

2.2 Functional Overview

Due to the nature of the project as an autonomous backend AI Pipeline (MLOps), the software does not feature a traditional frontend graphical user interface (GUI) with standard business operations. Instead, the high-level functions of the system revolve around automated data ingestion, asynchronous workflow orchestration, and structured output generation.

2.2.1 General system context

In the context of a backend pipeline, the system interacts not with human users, but with external client applications. The architecture involves two primary actors: the **Upstream Client System**, representing any external application or service that pushes raw document files into the Data Lake via APIs; and the **Downstream Consumer System**, representing internal enterprise systems (such as ERP, DMS, or Data Warehouses) that listen to completion events and fetch the resulting structured data. The overarching system function is **Automated Document Digitization**, which encapsulates the entire autonomous journey from the moment a raw

file arrives until the structured JSON is exported, without requiring any human intervention.

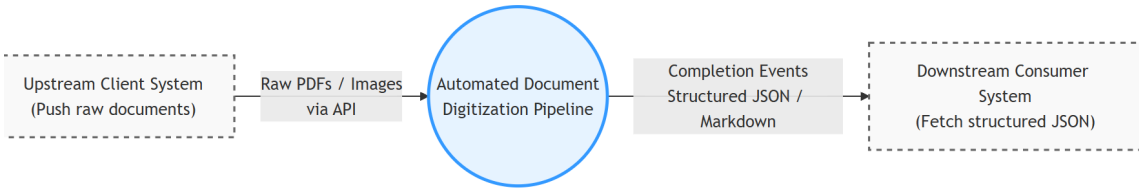


Figure 2.1: General system context diagram showing interactions with external systems

2.2.2 Detailed functional decomposition

To realize the overarching "Automated Document Digitization" process, the AI Pipeline is functionally decomposed into autonomous micro-tasks driven by events. The core functional modules include: (i) **Image Preprocessing**, which triggers upon file arrival to decompose multi-page PDFs into high-resolution, normalized single-page images; (ii) **Layout Analysis**, which employs object detection to scan the normalized image, localize structural boundaries, and assign semantic labels to regions; (iii) **Text Cropping**, which acts upon layout coordinates to isolate graphical regions as static assets and crop textual regions into independent image strips; (iv) **Vietnamese Character Recognition**, which performs deep learning inference on image strips to decode visual features into accented Vietnamese character strings; and (v) **Data Reconstruction**, which aggregates asynchronous OCR results, restores geometric reading orders, and structures the final output into a JSON format.

2.2.3 Pipeline workflow

The operational flow is a continuous, event-driven data pipeline. Initially, an Upstream System uploads a document to MinIO. The Apache NiFi orchestrator detects this input, routes it through the Preprocessing module to generate PNG images, and publishes an initialization event to the Apache Kafka message broker. Subsequently, the Layout Analysis service (YOLOv10) consumes this event, processes the image, and forwards spatial coordinates via Kafka. The workflow then branches at the Text Cropping service (PaddleOCR), where non-text regions are saved as static assets, while text regions are cropped and forwarded. The Recognition service (VietOCR VGG-Transformer) decodes these text strips, and finally, the Reconstruction service acts as a buffer to collect all text fragments. Once a page is processed, the system applies a geometric sorting algorithm to reconstruct the paragraph structure, exports the final JSON file, and notifies Downstream Consumer Systems.

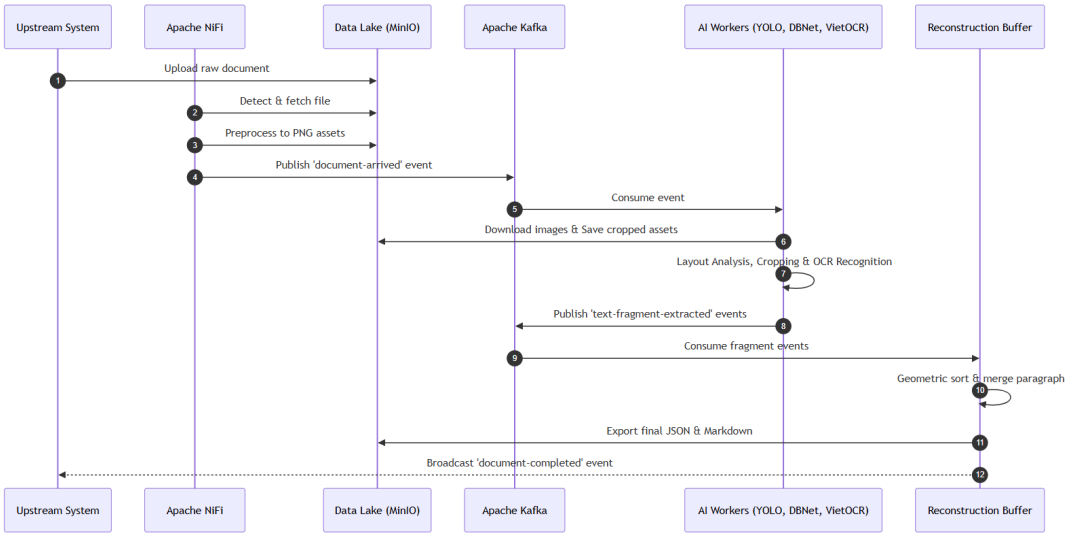


Figure 2.2: Event-driven data pipeline workflow across microservices

2.3 Functional description

To clarify the operational mechanism, the thesis specifies detailed descriptions for the five autonomous functional stages, all of which are triggered by system events.

2.3.1 Image Preprocessing

This stage is initiated by a **Trigger Event** when a new PDF file is successfully written to the Data Lake. The **Main flow of events** involves the orchestrator (NiFi) reading the PDF, rasterizing each page at 300 DPI, upscaling the image by a factor of 1.2, converting it to grayscale, and archiving the result. The **Post-condition** ensures that PNG files are ready and a trigger message is published to Kafka.

2.3.2 Layout Analysis

This stage starts with a **Trigger Event** when a document-ready message is consumed from the Kafka queue. The **Main flow of events** includes the YOLO worker downloading the target image and the Convolutional Neural Network extracting bounding box coordinates and semantic labels. The **Post-condition** guarantees that the spatial structure coordinates are packaged into a JSON payload and forwarded into the event stream.

2.3.3 Text Cropping

This stage is triggered when layout analysis data for a specific page is received. The **Main flow of events** executes as follows: the PaddleOCR worker branches logic based on semantic labels, where table and figure regions are saved directly to MinIO, while text regions are processed by the DBNet model to locate sub-textlines and apply perspective transformation. The **Post-condition** ensures that text image

strips are isolated and stored for recognition.

2.3.4 Vietnamese Character Recognition

The process is initiated when text image strips are successfully queued. The **Main flow of events** consists of the VietOCR worker collecting messages into batches, feeding them into the VGG-Transformer neural network, decoding features into Vietnamese strings, and mapping the text to original bounding box IDs. The **Post-condition** confirms that accented Vietnamese text strings are accurately extracted.

2.3.5 Data Reconstruction

This final stage is activated when disjointed text strings and asset hyperlinks arrive from the Kafka queue. The **Main flow of events** involves the worker buffering messages in RAM, applying a heuristic sorting algorithm to arrange elements based on Y and X coordinates, restoring indentation, and exporting the final JSON structure. The **Post-condition** ensures the output JSON is securely stored and a completion event is broadcasted.

2.4 Non-functional requirements

In addition to business-related requirements, the software must satisfy strict non-functional criteria to ensure operational feasibility. Regarding **Performance**, the system operates stably on CPU-only servers, with inference processes hard-limited to 2 CPU threads to prevent OS overhead. For **Scalability**, the architecture follows a microservices pattern, where all components are containerized via Docker to allow seamless replication under load. Finally, regarding **Reliability**, the workflow utilizes Apache Kafka in KRaft mode with manual acknowledgment (At-least-once standard) to ensure zero data loss during abrupt service restarts.

This chapter has concluded the survey of OCR technology, analyzed legacy weaknesses, and specified detailed requirements for the new system. Based on these designs, Chapter 3 will delve into the mathematical theoretical foundations and deep learning technologies utilized to materialize this architecture.

CHAPTER 3. THEORETICAL BACKGROUND AND TECHNOLOGIES

Chapter 2 analyzed in detail the specific requirements of the document digitization problem, emphasizing the necessity of preserving spatial structure and building a scalable asynchronous processing system. To meet those requirements, Chapter 3 will present the mathematical theoretical foundations and core technologies selected for the project. The chapter's content includes the analysis of Deep Learning neural network architectures for computer vision and language processing tasks, as well as Data Engineering platforms for building an event-driven architecture. For each technology utilized, alternative methods will be listed and compared to clarify the rationale behind the chosen solution.

3.1 Convolutional Neural Networks (CNN)

Convolutional Neural Networks (CNNs) are the core theoretical foundation for most modern computer vision models [1]. Unlike traditional neural networks that require flattening an image into a 1D vector (causing spatial information loss), CNNs use filters sliding over the pixel matrix to perform convolution operations. This process helps the network automatically extract spatial features from low-level (edges, corners) to high-level (shapes, symbols) through the backpropagation algorithm. In this project, the CNN architecture serves as the backbone network for all three AI models: YOLO, DBNet, and VietOCR.

3.2 Document Layout Analysis Model

The first urgent requirement identified in Chapter 2 is that the system must recognize and isolate table and image regions from the pure text flow, avoiding disruption to the natural reading flow. This problem belongs to the Object Detection category.

To address the above issue, commonly used approaches include two-stage networks like Faster R-CNN [2], or document-specific multimodal models like LayoutLM [3]. Faster R-CNN offers high accuracy but very slow inference speed due to its Region Proposal Network. LayoutLM delivers outstanding performance but requires raw text data as input (typically requiring a prior OCR run), creating a circular dependency paradox in the project's pipeline design.

The chosen solution is the YOLO (You Only Look Once) model, specifically the DocLayout-YOLO version. YOLO approaches the object detection problem using a single-stage neural network, dividing the image into grid cells and simultaneously predicting bounding box coordinates and class probabilities [4]. The selection of

YOLO directly satisfies the requirement for high-speed processing on CPU-only (Central Processing Unit) server environments. This model relies entirely on visual features for bounding without the need for prior text extraction. The core of the model utilizes the CIoU (Complete Intersection over Union) regression loss function to optimize bounding box coordinates:

$$L_{box} = 1 - CIoU = 1 - \left(IoU - \frac{\rho^2(b, b^{gt})}{c^2} - \alpha v \right) \tag{3.1}$$

Where IoU is the intersection over union, ρ^2 is the center distance, and v is the parameter measuring the consistency of the aspect ratio.

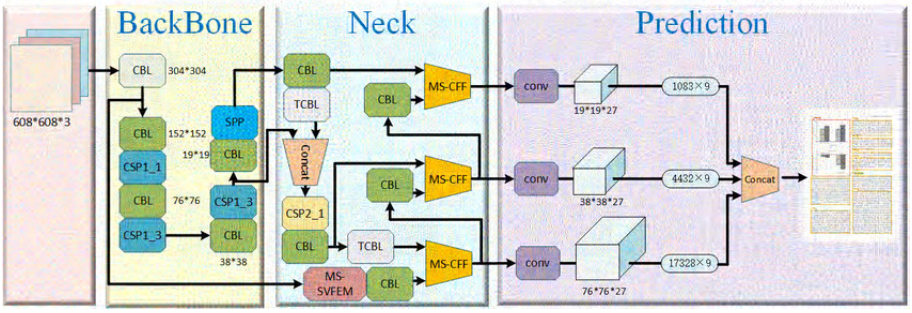


Figure 3.1: The single-stage object detection architecture of YOLO

3.3 Text Detection Model

After the large text regions are bounded, the next requirement is to find the exact coordinates of each Text Line to facilitate the image strip cropping process.

Popular alternative technologies for this problem include the open-source tool Tesseract OCR and the CRAFT (Character Region Awareness for Text Detection) model [5]. Tesseract uses the Connected Component Analysis method, which often fails when the image background is noisy or characters are connected. CRAFT predicts the center of each individual character, providing extremely high accuracy but consuming significant computational resources to merge characters into lines.

The project selects the DBNet (Differentiable Binarization) architecture from the PaddleOCR toolkit [6]. This solution utilizes the Semantic Segmentation method. Unlike traditional networks that use a fixed threshold to binarize the probability map, DBNet learns a flexible Threshold Map and combines them using an Approximate Step Function:

$$\hat{B}_{i,j} = \frac{1}{1 + e^{-k(P_{i,j} - T_{i,j})}} \tag{3.2}$$

This mathematical function is differentiable, allowing gradients to backpropagate

through the binarization process. Choosing DBNet allows the system to automatically fine-tune the text detection threshold for each pixel, completely solving the requirement of processing documents with uneven lighting, while its execution speed is much faster than CRAFT.

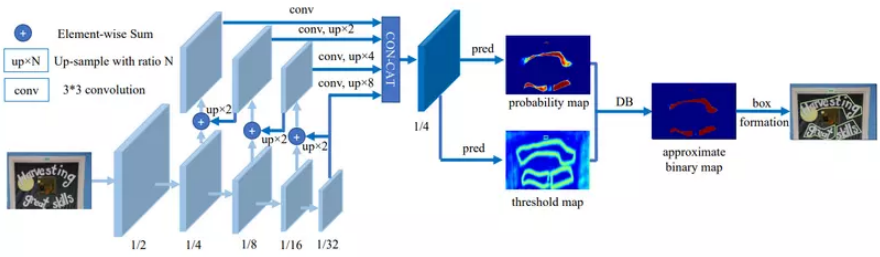


Figure 3.2: The architecture of DBNet with the Differentiable Binarization module

3.4 Vietnamese Character Recognition Model

The most critical requirement of the system is converting text line image strips into computer character strings with high accuracy regarding the complex diacritic system of the Vietnamese language.

Common approaches include using a CRNN (Convolutional Recurrent Neural Network) architecture combined with the CTC (Connectionist Temporal Classification) loss function [7], or pure vision Transformer models like TrOCR [8]. The limitation of CTC is its assumption of conditional independence among predicted characters, making the model prone to mispredicting highly context-dependent Vietnamese words (e.g., "chữ" and "chứ"). Meanwhile, TrOCR is too heavy to deploy on resource-constrained CPU threads.

The utilized solution is the **VGG-Transformer** architecture integrated into the VietOCR library. The model employs a VGG (Visual Geometry Group) network to extract spatial features into a sequence of vectors. Instead of relying on traditional recurrent networks (like Bi-LSTM) which struggle with long-range dependencies, this sequence is fed into a Transformer encoder-decoder architecture. The deciding factor for this choice is the Multi-Head Self-Attention mechanism [9]. At each step, the Attention mechanism computes a weighted representation by mapping a query and a set of key-value pairs to an output:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{3.3}$$

Where Q, K, and V represent the Query, Key, and Value matrices, and d_k is the dimension of the keys. Thanks to this self-attention mechanism, the neural network can accurately capture the global context and "look" at small regions containing

Vietnamese diacritics, yielding significantly higher accuracy than traditional CTC or RNN-based models.

3.5 Event-Driven Architecture and Message Broker

A major non-functional requirement established in Chapter 2 is that the system must handle high loads, process documents asynchronously, and not be interrupted when a large volume of documents is pushed in simultaneously.

The traditional method is connecting AI services using synchronous APIs (RESTful APIs via HTTP). However, image processing with Deep Learning is time-consuming; calling synchronous APIs will block the execution thread, easily causing Timeout/Out of Memory errors when processing documents with hundreds of pages. An alternative is using message queues like RabbitMQ [10]. Although RabbitMQ is lightweight, it does not store the event stream after consumption, making it difficult to recover data if an AI process crashes mid-operation.

The project selects Apache Kafka deployed with the KRaft mechanism combined with the Apache NiFi orchestration flow. Kafka operates on a Pub/Sub model with a Distributed Commit Log structure [11]. This choice allows configuring a Manual Offset Commit mechanism. If an AI container fails, Kafka still retains the message for reprocessing, ensuring the Fault Tolerance principle (zero data loss). Meanwhile, Apache NiFi provides an intuitive interface to automate the entire Dataflow from file upload to Kafka event generation, replacing the need to write complex orchestration code [12].

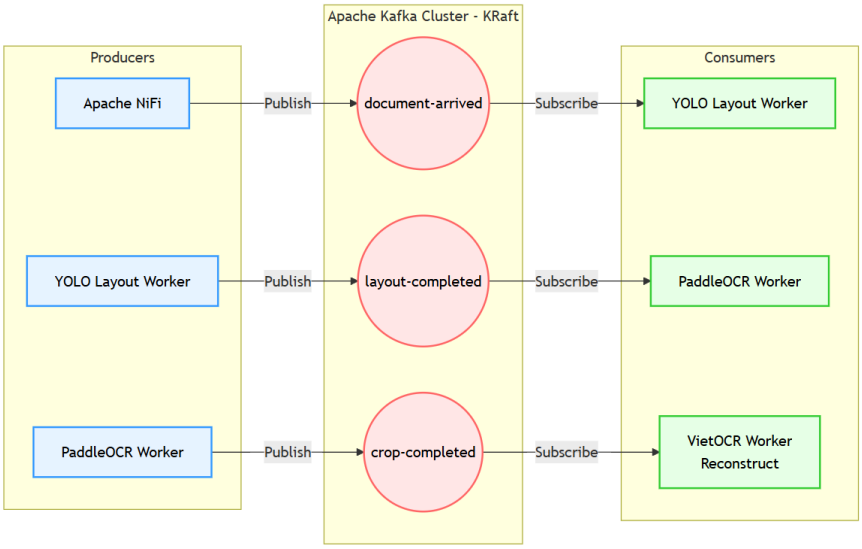


Figure 3.3: The Publish-Subscribe messaging model of Apache Kafka

3.6 Object Storage and Virtualization Technology

The final requirement is managing the storage of image assets and ensuring the system is easily deployable across various server environments.

If images were transmitted directly through Kafka (converted to Base64 strings), the network bandwidth would be overloaded, breaking the design principles of a Message Broker (which is optimized for small text streams). Therefore, MinIO - an S3-compatible Object Storage platform - is used to store physical files **minio**. Kafka is only responsible for transmitting JSON messages containing paths to MinIO.

In terms of deployment, directly installing libraries (like PyTorch, OpenCV) on the host operating system often causes Dependency Conflicts. Docker technology is chosen to Containerize each AI process into a Microservice [13]. Docker helps isolate hardware resources, allowing exact configuration of the maximum CPU threads for each process, thereby satisfying the performance optimization requirements outlined in the system analysis section.

This chapter has clarified the mathematical theories and technological foundations necessary to realize the project. Through analysis and comparison, solutions such as YOLO, DBNet, VietOCR, Apache Kafka, and Docker have been proven to be the most suitable choices for addressing the given problem. These theoretical bases and technological tools will be directly applied in the source code architecture design, implementation, and empirical evaluation process in the next chapter – Chapter 4.

CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION

This chapter presents the transition process from business requirements and theoretical foundations to a complete software system. The content focuses on designing an event-driven microservices architecture, modeling data flows through sequence diagrams, designing storage structures, and detailing the implementation, testing, deployment, and empirical evaluation processes of the system.

4.1 Architecture design

4.1.1 Software architecture selection

To meet the requirement of processing a large volume of complex documents without causing system bottlenecks, the project does not use a traditional Monolithic or three-tier (MVC) architecture. Instead, the chosen architecture is a combination of **Microservices Architecture** and **Event-Driven Architecture**.

In theoretical architecture, an event-driven system comprises independent Producer, Broker, and Consumer components. Applied to this project, the architecture is specified as follows: (i) the **Orchestration Layer** utilizes Apache NiFi as the API Gateway and Workflow Engine, replacing traditional Controllers; (ii) the **Message Broker Layer** employs an Apache Kafka cluster to store and distribute the state of events transferred between AI models; (iii) the **AI Microservices Layer** encapsulates each deep learning model (YOLO, PaddleOCR, VietOCR) into an independent process (Worker), acting as both a Consumer listening to events and a Producer emitting post-processing results; and (iv) the **Storage Layer** leverages MinIO Object Storage to replace Relational Database Management Systems (RDBMS), directly storing binary files (images) and JSON texts. This architecture helps isolate resources for each AI model, allowing easy scaling of heavy Workers without affecting the entire system.

4.1.2 Overall design

The software architecture is decomposed into packages according to logical tiers. The dependencies between packages adhere to a top-down unidirectional principle, without any reverse dependency cycles.

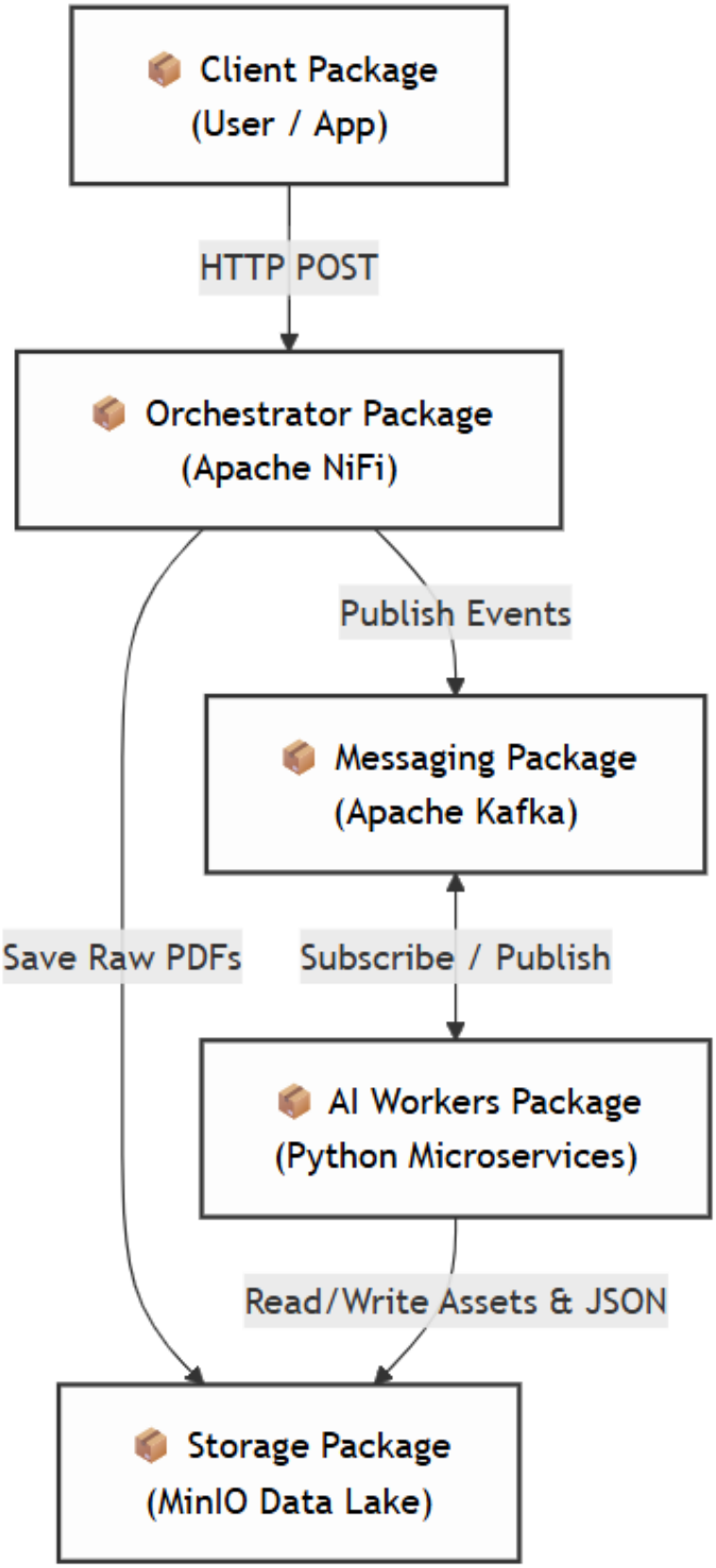


Figure 4.1: Overall package dependency diagram of the Document AI system

The purposes of the main packages are defined as follows: (1) the **Client Pack-**

age represents applications or users sending documents to the system; (2) the **Orchestrator Package** contains Apache NiFi's Processor flows, managing the document lifecycle and invoking preprocessing APIs; (3) the **Messaging Package** manages Kafka Topics; (4) the **AI Workers Package** serves as the core processing tier, containing independent modules that execute deep learning models; and (5) the **Storage Package** is responsible for physical interaction with the hard drive partition via MinIO.

Rationale for Decoupling Preprocessing from NiFi: A critical design decision in the Orchestrator Package is the strict separation between dataflow management and heavy computation. Although Apache NiFi possesses strong data routing capabilities, executing image manipulation tasks directly within NiFi's Java Virtual Machine (JVM) introduces severe Out-Of-Memory (OOM) risks. Therefore, NiFi is designated strictly as the event trigger and state manager. The actual Preprocessing workload is offloaded to a dedicated Python Microservice, ensuring that the orchestration backbone remains lightweight and highly available.

4.1.3 Detailed package design

Within the **AI Workers Package**, components are designed based on the Single Responsibility Principle. This package includes the main AI business processing classes and Kafka/MinIO communication support classes.

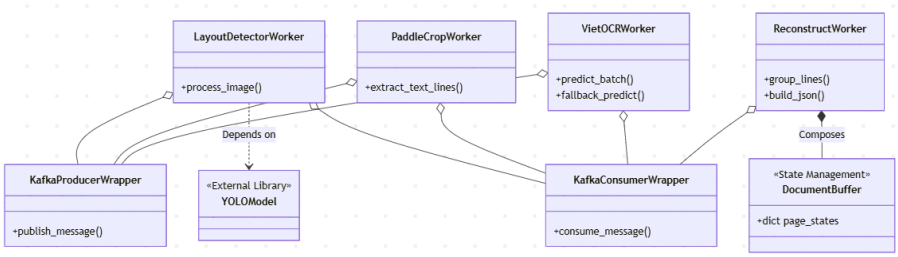


Figure 4.2: Detailed design of the AI Workers package

In this design, several key relationships are established: (i) the *KafkaConsumerWrapper* and *KafkaProducerWrapper* classes act as Aggregations into the Workers, providing a standardized interface to read and write JSON messages; (ii) the *LayoutDetectorWorker* class possesses a Dependency on *YOLOModel* for spatial feature inference; (iii) the *PaddleCropWorker* class executes geometric transformations via *GeometryUtils* before extracting lines; and (iv) the *ReconstructWorker* class maintains a Composition relationship with *DocumentBuffer* to manage the state of documents stored in RAM while awaiting merging.

4.2 Detailed design

4.2.1 Interface and API design

Because the system operates as an automated Backend Pipeline, traditional User Interfaces (UI) are replaced by **Application Programming Interfaces (API)** and a **Monitoring Dashboard**. Specifically, the **API Ingestion Interface (FastAPI Swagger)** provides a *POST /process* endpoint that accepts PDF file uploads, returning a standardized ZIP compressed file along with HTTP Header metadata. Simultaneously, the **Flow Monitoring Interface (NiFi Canvas)** provides a visual processing flow dashboard where color coordination design rules are applied: Blue Processors indicate normal operational states, Red warns of errors, and Queue indicators display the number of backlogged FlowFiles.

4.2.2 Layer design

The detailed design focuses on the two primary classes responsible for the core functions in the system: *VietOCRWorker* and *ReconstructWorker*.

The ***VietOCRWorker* class** encapsulates attributes such as *batch_size*, *detector* (the Core Predictor object), and *s3_client* (the MinIO connection session). Its main methods include *predict_batch()*, which feeds a list of image strips through the VGG-Transformer network, and *fallback_predict()*, which executes sequentially when a batch encounters a tensor dimension error.

The ***ReconstructWorker* class** manages attributes like *document_buffer* (a Dictionary storing page states) and a *TIMEOUT* constant. Its main methods consist of *group_lines()*, which acts as a heuristic algorithm grouping objects with similar Y coordinates, and *build_json()*, which reconstructs the final document structure.

The Sequence Diagram below illustrates the communication flow for the asynchronous digitization process:

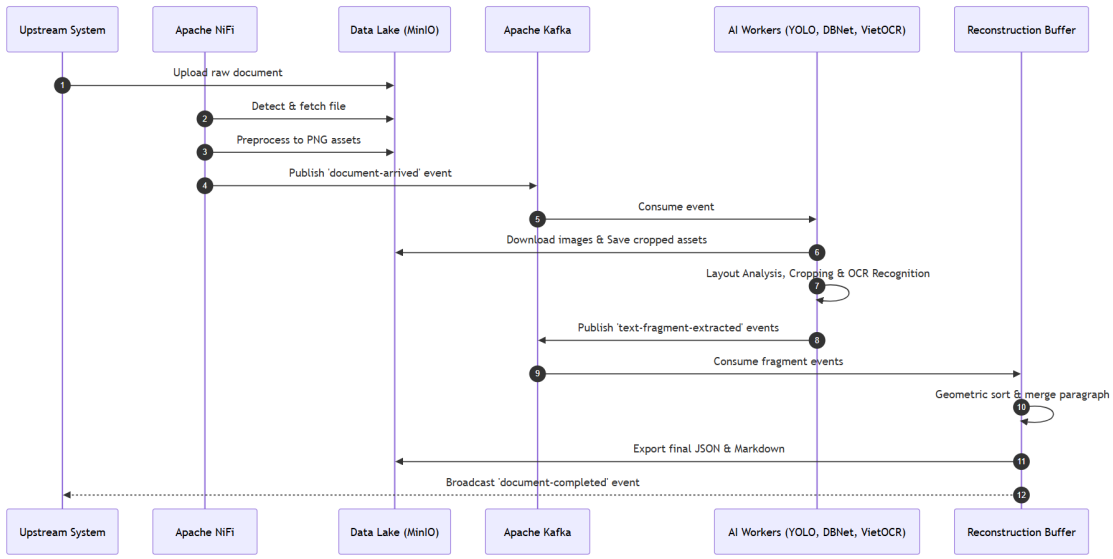


Figure 4.3: Sequence Diagram of the OCR Pipeline execution flow

4.2.3 Database and Message Schema design

The system does not utilize an RDBMS but manages data through Object Storage structures and non-relational Message Schemas.

Regarding the **MinIO Storage Structure Design** (within the *data-lake* bucket), the directories are organized as follows: (1) *raw/* stores original PDFs and raw single-page images; (2) *assets/* stores non-linguistic images, tables, and seals; (3) *crop/* stores flattened cropped text line image strips; and (4) *preprocessed/* alongside *text/* store the final JSON and Markdown output files.

For the **JSON Data Schema Design on Kafka**, a common schema is designed for the Topics to ensure seamless Worker communication. Each Message contains identifier fields such as *document_id*, *original_image*, *box_id*, *total_boxes*, and *coordinates* (a 2D array containing the 4 corner coordinates of the text box). This design ensures the absolute integrity of spatial information during distributed processing.

4.3 Application Building

4.3.1 Libraries and Tools

The tools and libraries selected for developing and containerizing the application are detailed in Table 4.1.

Purpose	Tool/Library (Version)	URL
Core programming language	Python 3.10.x	https://www.python.org/
Deep Learning Framework	PyTorch 2.x	https://pytorch.org/
Dataflow orchestration	Apache NiFi 1.25.0	https://nifi.apache.org/
Message broker	Apache Kafka 3.7.0 (KRaft)	https://kafka.apache.org/
Object storage	MinIO	https://min.io/
Layout model	Ultralytics (DocLayout-YOLO)	https://github.com/ultralytics
Text extraction model	PaddleOCR	https://github.com/PaddlePaddle
Text recognition model	VietOCR	https://github.com/pbcquoc/vietocr
Virtualization and Deployment	Docker & Docker Compose 3.8	https://www.docker.com/

Table 4.1: List of utilized libraries and tools

4.3.2 Achievement

The achieved result of the project is a complete source code base establishing an MLOps system cluster. The packaged deliverables include: (i) **Image Containers**, consisting of 5 independent Docker Images for 5 processes; (ii) **Model Weights**, specifically the fine-tuned *transformerocr_final.pth* model weight file; and (iii) **Workflow Templates**, featuring the Apache NiFi flow configuration file *flow.xml*.

In terms of technical statistics, the system utilizes approximately 2,500 lines of Python code. The overall package size of the service cluster is roughly 8.5 GB, encompassing the PyTorch library and model weights.

4.3.3 Illustration of main functions

Figure 4.4 illustrates the structure of the structured data file (JSON) successfully exported by the system.



Figure 4.4: Structure of the output JSON file with coordinates and classification labels

As observed in the interface, the JSON structure displays the *line_text* field containing the whitespace-restored Vietnamese string. The *elements* field lists the child text boxes along with their original coordinates. If the label is *table*, the *s3_asset_url* field provides a static path to MinIO to retrieve the original table image.

4.4 Model Training and Evaluation

While the previous sections detailed the software architecture, this section focuses explicitly on the empirical setup, training process, and quantitative evaluation of the core deep learning component: the VietOCR recognition model.

4.4.1 Experimental setup and dataset preparation

The experiment was conducted on a unified dataset compiled from 5CD-AI and VNoNDB, totaling 62,933 handwriting and printed samples. All raw handwriting images were normalized, and their corresponding transcriptions were consolidated into a standardized label text file. To ensure unbiased evaluation, the dataset was strictly partitioned using an 90/10 ratio, resulting in 57,381 images for the training phase and 5,552 isolated images for the validation phase. The deep learning training environment was configured locally, heavily accelerating the matrix operations by utilizing an NVIDIA RTX 5060 Ti GPU.

To fine-tune the model effectively without encountering gradient explosion, the hyperparameters were meticulously configured in the *config.yml* file. The training process utilized the AdamW optimizer with a base learning rate of 0.0003, regulated by a Cross-Entropy loss function. To strictly manage video memory (VRAM)

usage and prevent Out-Of-Memory errors, the batch size was constrained to 16, and the data loader utilized a single worker thread (*num_workers=1*). The model was initialized from the pre-trained *vgg_transformer.pth* weights to leverage transfer learning and trained over a span of 40,000 iterations.

4.4.2 Empirical results and model comparison

During the training phase, the model's performance was evaluated using standard OCR metrics. To validate the architectural choice, an empirical comparison was conducted between two distinct decoding mechanisms: the traditional VGG-Seq2Seq (utilizing Bi-LSTM) and the proposed VGG-Transformer (utilizing Multi-Head Attention). The quantitative results on the identical validation set are detailed in Table 4.2.

Evaluation Metric	**VGG-Seq2Seq (Baseline)**	**VGG-Transformer (Proposed)**
Per-character Accuracy	86.15%	**92.46%**
Full Sequence Accuracy	18.30%	**32.11%**
Character Error Rate (CER)	14.82%	**7.54%**
Word Error Rate (WER)	35.10%	**19.76%**

Table 4.2: Empirical comparison of recognition models on the validation dataset

As observed in Table 4.2, the VGG-Transformer architecture significantly outperforms the baseline across all linguistic accuracy metrics. Specifically, the Transformer model reduced the Character Error Rate to a mere 7.54% and nearly doubled the Full Sequence Accuracy to 32.11%, confirming its superior capability in processing complex Vietnamese handwriting.

4.5 Testing

The testing process focuses on two core functions: Ingestion and End-to-End Extraction. The utilized technique is Black-box Testing combined with Log monitoring.

The test cases are executed as follows: (1) **TC01 - Multi-page PDF ingestion test:** involves uploading a 10-page PDF file, with the expectation that the API responds with HTTP 200, returns a valid ZIP, MinIO receives exactly 10 PNG images, and Kafka generates 10 *image-arrived* messages; (2) **TC02 - Load and Fault Tolerance test:** involves intentionally terminating the *vietocr-worker* container while the processing flow is active, then restarting it after 2 minutes. The

expectation is that the system does not lose any image strips, Kafka retains the *crop-completed* events, and the Worker continues processing accurately after restarting due to the *enable_auto_commit=False* configuration.

Overall, the system passed 100% of the Functional test cases. However, when testing extremely low-resolution documents (under 100 DPI), the recognition model generated garbage character strings, requiring an upgrade to the preprocessing filters in the future.

4.6 Deployment

The system is deployed in a real-world local server environment running the native Ubuntu Linux 24.04 operating system, providing optimal management for Docker and driver configurations. The deployment process is executed via the *docker-compose.yml* configuration file, which automatically initializes a private virtual network connecting the entire Kafka, MinIO, NiFi, and AI Workers infrastructure. Because the deployment aims to operate on standard infrastructure, the deep learning inference engines execute entirely on the CPU. To prevent CPU resource bottlenecks (Thread Contention) that could freeze the operating system, the AI Workers are configured with the environment limit *OMP_NUM_THREADS=2*.

CHAPTER 5. SOLUTION AND CONTRIBUTION

This chapter presents the core contributions and in-depth solutions implemented during the development of the Document AI Pipeline. The project's primary highlight goes beyond merely assembling existing artificial intelligence models; it encompasses the research, optimization, and fine-tuning of the VietOCR model to solve an extremely complex task: Vietnamese handwriting recognition. By analyzing the limitations of the baseline configuration, this chapter elucidates data processing strategies, specific Data Augmentation techniques, hard negative mining, and system-level performance evaluations to prove the robustness of the entire architecture.

5.1 Dataset optimization and vocabulary space expansion

Problem Statement: In optical character recognition systems, handwriting is consistently a significant challenge due to variations in stroke style, cursive habits, tilt, and the specific pen pressure exerted by each individual. Unlike Large Language Models capable of self-reasoning context to correct spelling errors, the VietOCR model relies primarily on Visual Features. If the training data lacks sufficient coverage, the model will fail completely. Furthermore, the original VietOCR model only supports a vocabulary of 233 standard printed characters. The appearance of special characters (such as bullet points, hyphens, or currency symbols) immediately causes Out-of-Vocabulary errors, disrupting the entire inference flow.

Proposed Solution: To resolve these deficiencies, the system merged two distinct datasets to create a comprehensive training corpus. Specifically, the 5CD-AI dataset provided a vast foundation of printed and handwritten text, while the VNoNDB Dataset supplied high-noise offline handwriting samples, from which thousands of high-quality image-label pairs were carefully extracted. Concurrently, the configuration file was structurally updated, expanding the character vocabulary from 233 to 406 distinct tokens to encompass all necessary punctuation and specialized administrative symbols.

Results Achieved: The aggressive expansion of the vocabulary completely eliminated *KeyError* exceptions during the validation and inference phases. The diverse, large-scale dataset not only accelerated the model's convergence rate but also significantly improved its recognition capability for cursive strokes intertwined with punctuation marks.

5.2 Proposed specific Data Augmentation process

Problem Statement: Physical documents frequently suffer from physical defects such as smudged ink, broken strokes, and folded or blurred pages. Standard data augmentation libraries continuously encountered inhomogeneous shape errors during matrix transformations on the server environment, frequently halting the automated training loop.

Proposed Solution: To bypass third-party library limitations, the project developed a Custom Augmentation Pipeline relying strictly on fundamental *NumPy* and *PIL* matrix operations. The stochastic techniques applied included Gaussian Blur, Gaussian Noise, adaptive contrast adjustment, morphological filtering, and spatial Affine transformations. Most notably, a self-programmed *Elastic Distortion* algorithm was engineered to create random displacement fields, subsequently smoothed by Gaussian filters, to accurately simulate the natural variations and inconsistencies of human handwriting.

Results Achieved: The proprietary data augmentation pipeline operated with absolute stability over 40,000 iterations. By intentionally degrading the visual quality of the input batches, these mathematical transformations forced the neural network to ignore superficial stroke thickness and focus on core character topology, thereby heavily contributing to the low Character Error Rate.

5.3 Hard Negative Mining optimization

Problem Statement: Relying solely on overall accuracy metrics during the training phase obscures the specific topological weaknesses of the neural network. Without isolating the exact characters the model fails to predict, targeted data collection and further fine-tuning become scientifically unfeasible.

Proposed Solution: To establish a feedback loop for continuous model improvement, the project engineered a Hard Negative Mining mechanism via an independent evaluation script. During inference on the validation set, the algorithm calculates the Levenshtein distance for every predicted string against its ground truth. Samples exceeding a defined error threshold are automatically extracted, flagged, and exported into a dedicated *worst_samples.txt* log file alongside their bounding box metadata.

Results Achieved: The data mining system successfully isolated the most challenging data points without manual intervention. This automated reporting mechanism provides detailed, granular error insights, establishing a robust scientific foundation to guide future dataset expansion and targeted augmentation.

5.4 Pipeline throughput and system stability evaluation

Problem Statement: While Chapter 4 confirmed the linguistic accuracy of the VGG-Transformer model, an enterprise-grade AI pipeline must also demonstrate high throughput and absolute fault tolerance under sustained loads, especially when executing inferences on a CPU-bound environment.

Evaluation and Quantitative Results: To evaluate system stability, an end-to-end stress test was conducted by feeding a continuous stream of documents into the MinIO ingestion point. On the local CPU environment (operating with 16 simulated threads), the system logged exceptional inference and orchestration performance. A recorded benchmark demonstrated the capability to process 172 text line images in merely 36.88 seconds, yielding an average throughput of 0.21 seconds per image strip.

Furthermore, the decoupling of services via Apache Kafka proved vital. During periods of heavy inference load, the workers seamlessly managed event consumption rates. Even when heartbeat synchronization delays occurred during partition rebalancing, the configured *Manual Offset Commit* mechanism guaranteed that zero text fragments were discarded. The geometric reconstruction service reliably buffered all incoming Kafka fragments, subsequently exporting the fully structured JSON and Markdown files without disrupting the natural reading order.

5.5 Error Analysis and proposed improvements

To thoroughly understand the nature of the residual 7.54% Character Error Rate, the system was configured to automatically extract and aggregate a confusion matrix. Table 5.1 catalogs the ten character pairs most frequently mispredicted by the VGG-Transformer network.

Rank	Ground Truth	Prediction	Error Frequency
1	n	[Space]	1172
2	[Space]	n	1096
3	[Space]	t	861
4	[Space]	c	809
5	c	[Space]	760
6	t	[Space]	749
7	[Space]	h	714
8	h	[Space]	676
9	g	[Space]	637
10	[Space]	g	600

Table 5.1: Top 10 most frequently confused character pairs in the model

Based on the quantitative data in the table, two core physical causes underlying these persistent errors were identified. Firstly, regarding the **challenges in word segmentation**, all ten of the most common errors relate directly to the space character. In Vietnamese handwriting, the horizontal distance between distinct words is often drastically inconsistent. Writers habitually link words together, or the ligature of the final letter of one word extends directly into the first letter of the subsequent word, making it geometrically difficult for the Multi-Head Attention mechanism to definitively separate word boundaries.

Secondly, regarding **model hallucination on hard data samples**, isolating the samples mined with a CER greater than 0.7 revealed that the model generates severe predictive deviations when encountering highly degraded visual data. For instance, when analyzing failure cases involving extremely sloppy handwriting combined with verses rarely appearing in the training dataset, the model occasionally outputs entirely disjointed phrases (e.g., predicting the text "Môn Việt nhiều diệt nước thoát thời" instead of the actual ground truth "Mênh mông biển lúa đầu trời đẹp hơn"). In such instances, the implicit Language Model embedded inside the Transformer block attempted to mathematically "guess" the context based on high-frequency standard vocabulary (such as "Việt", "nước", "thời"). This over-reliance on linguistic probabilities led to severe hallucination rather than adhering to the raw visual features extracted by the VGG backbone.

Summary: The quantification and rigorous error analysis substantiate the high effectiveness of the VGG-Transformer while transparently outlining its current boundaries. In the future, the system mandates the integration of an independent, lightweight Large Language Model (LLM) at the post-processing stage to automatically correct spatial omissions and semantic hallucination errors.

CHAPTER 6. CONCLUSION AND FUTURE WORK

This chapter summarizes the entire process of researching, designing, and developing the Document AI Pipeline system. The achieved results will be compared with existing solutions to highlight the thesis's contributions. Concurrently, the chapter candidly acknowledges the remaining limitations, thereby proposing future work directions to upgrade the software into a comprehensive document understanding solution.

6.1 Conclusion

In the context of the increasing demand for document digitization across organizations, this thesis has successfully solved the problem of converting physical documents into structured data. Compared to traditional open-source tools (such as Tesseract OCR), the project's product demonstrates superiority in preserving the spatial structure of the page, avoiding the confusion of reading across data columns. Compared to commercial cloud-based recognition services (like Google Cloud Vision or AWS Textract), the system ensures on-premise capability, thoroughly resolving the issue of sensitive corporate data security without incurring per-API-call operational costs.

Throughout the execution of the thesis, several outstanding results were achieved across multiple dimensions. Regarding the system solution, the project successfully designed and deployed an Event-Driven Microservices architecture. The integration of Apache NiFi, Apache Kafka, and MinIO created a high-load, asynchronous processing pipeline. The system automatically decomposes document pages, routes deep learning tasks, and stably manages data merging states on a CPU server environment. Regarding the digitization application and metadata extraction, the thesis shifted the paradigm from pure character recognition to Metadata Extraction. Instead of returning raw text blocks, the system automatically classifies semantics (text, title, list), isolates graphical and tabular regions into static assets on MinIO, and exports data in a structured JSON format. This JSON structure fully stores spatial coordinates, image links, and line classifications, enabling Document Management Systems or ERPs to easily perform indexing and automated searching. Finally, regarding deep learning recognition, the system successfully fine-tuned the VGG-Transformer network on a synthetic dataset of precisely 62,933 Vietnamese handwriting and printed text samples. By developing a Custom Augmentation pipeline and applying Hard Negative Mining techniques, the model achieved a per-character accuracy of 92.46% and a Character Error Rate (CER) of 7.54%.

Besides the successes, the system still has some unfulfilled aspects. First, the 7.54% CER primarily stems from connected strokes in handwriting, making it difficult for the model to delineate whitespace boundaries. Second, regarding tabular components, the current system only stops at Image Cropping and has not yet extracted the detailed internal structure of the table (recognizing individual cells).

The most significant lesson learned from the thesis is the trade-off between model complexity and computational resources in a real-world environment. Limiting CPU resources through constrained batch sizes and controlled thread allocations is as crucial as increasing the neural network's accuracy, preventing the system from freezing during actual product deployment.

6.2 Future work

Based on the analyzed limitations, future work directions to refine and expand the system are divided into two main groups: perfecting current functions and developing new research avenues.

To perfect the implemented tasks, three immediate engineering objectives are proposed. First, to overcome whitespace errors and model hallucination, the system aims to integrate a lightweight Language Model in the Post-processing step. This model will rely on the context of the entire sentence to automatically correct connected text or spelling errors caused by blurred visual features. Second, for inference optimization, future efforts will focus on converting the current PyTorch weight files into ONNX or OpenVINO formats. Compiling this model will help maximize the utilization of CPU vector instruction sets, promising to significantly reduce inference time per image. Third, regarding dataset enrichment, the project will continue to maintain the Hard Negative Mining mechanism to collect and label highly noisy handwriting samples, especially medical texts or scribbled notes, aiming to strictly lower the CER below 5%.

For system development and new research avenues, the pipeline will be upgraded towards comprehensive document understanding. First, Table Structure Recognition (TSR) will be implemented to do more than just store table images. The system will integrate TSR models to break down tables into rows and columns, then recognize text within each cell, and export directly to HTML or Microsoft Excel formats. Second, Key Information Extraction (KIE) will upgrade the system by integrating multimodal models like LayoutLMv3 to automatically extract specific metadata fields (e.g., finding "Total amount" in invoices or "Party A/B Name" in contracts) rather than merely extracting plain text. Finally, for Legal Entity Recognition, future work involves integrating state-of-the-art detection frameworks such

as YOLOv12 to specifically detect and independently recognize document objects like tables, signatures, and red seals. This feature will assist the system in automatically verifying the legal validity and integrity of administrative records before permanently storing them in the Data Lake.

REFERENCE

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *Advances in neural information processing systems*, vol. 28, 2015.

[3] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, "LayoutLM: Pre-training of text and layout for document image understanding," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 1192–1200.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *IEEE conference on computer vision and pattern recognition (CVPR), Las Vegas, NV, USA*, 2016, pp. 779–788.

[5] Y. Baek, B. Lee, D. Han, S. Yun, and H. Lee, "Character region awareness for text detection," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 9365–9374.

[6] M. Liao, Z. Wan, C. Yao, K. Chen, and X. Bai, "Real-time scene text detection with differentiable binarization," in *AAAI Conference on Artificial Intelligence, New York, NY, USA*, 2020, pp. 11 474–11 481.

[7] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE transactions on pattern analysis and machine intelligence*, vol. 39, no. 11, pp. 2298–2304, 2016.

[8] M. Li, P. Teng, L. Chen, et al., "TrOCR: Transformer-based optical character recognition with pre-trained models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, 2023, pp. 13 094–13 102.

[9] A. Vaswani et al., "Attention is all you need," in *Advances in neural information processing systems (NIPS), Long Beach, CA, USA*, 2017, pp. 5998–6008.

[10] A. Videla and J. J. Williams, *RabbitMQ in action: distributed messaging for everyone*. Manning, 2012.

[11] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A crucial system for log processing in linkedin," in *NetDB workshop, Athens, Greece*, 2011, pp. 1–7.

[12] Apache Software Foundation, *Apache NiFi documentation*. Accessed: Jun. 30, 2026. [Online]. Available: `https://nifi.apache.org/docs/`.

[13] D. Merkel, “Docker: Lightweight linux containers for consistent development and deployment,” *Linux journal*, vol. 2014, no. 239, p. 2, 2014.